Exorcism of the Haunted Codebase

Modernizing Our Oldest, Scariest Code with AI

Edith Harbaugh + Zach Davis

Enterprise AI Summit 2026

LaunchDarkly

LaunchDarkly at a Glance

Founded
2014

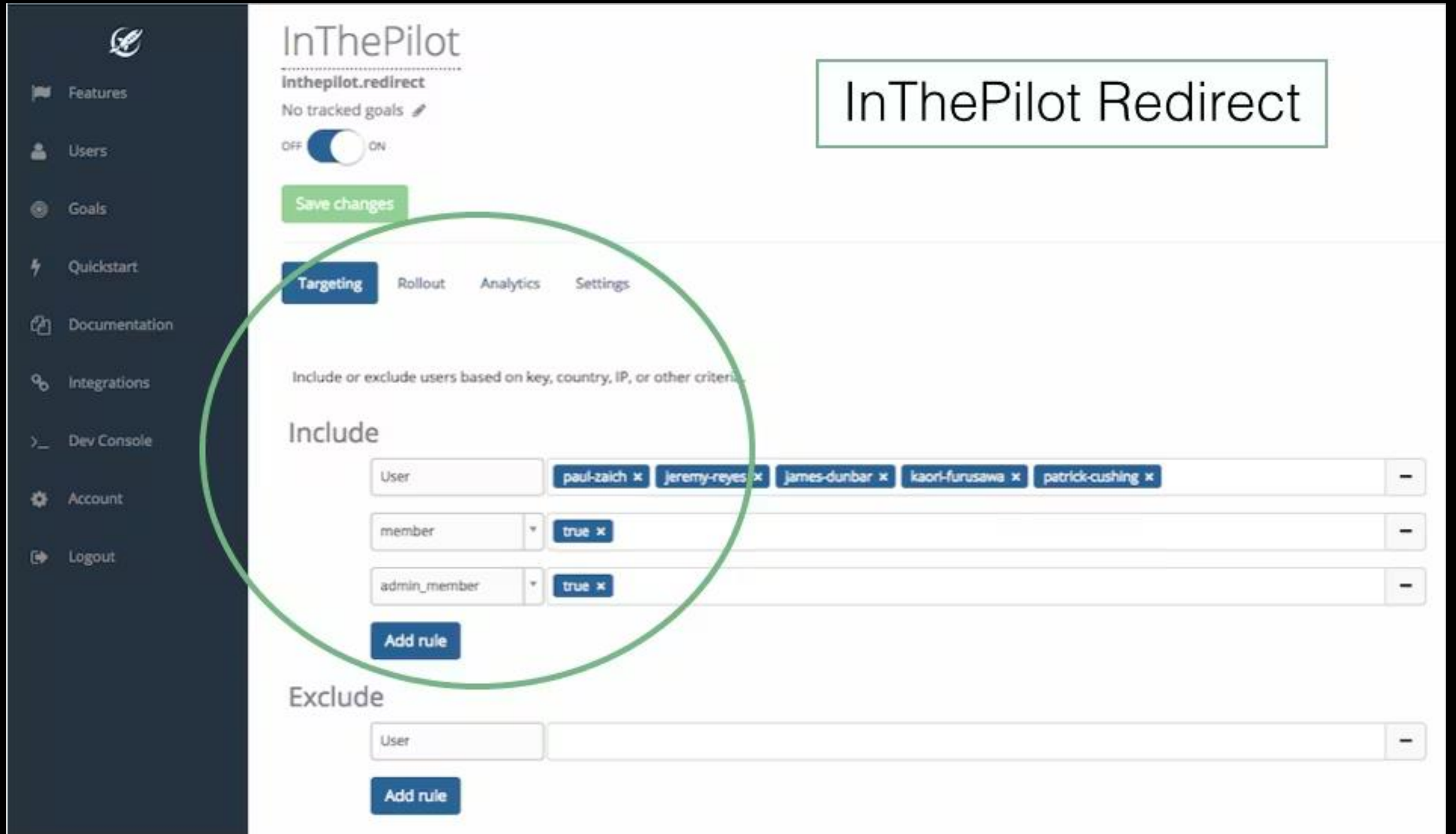
Building Go & JS/TS code for
12 years

5,500+
customers worldwide

45 trillion
feature flags evaluated daily

> Few things have fundamentally changed the product development lifecycle as much as LaunchDarkly . . . once you have it, you realize that you can't live without it.
>
> - “Working Code” podcast

LaunchDarkly


Features
Users
Goals
Quickstart
Documentation
Integrations
Dev Console
Account
Logout
InThePilot
inthepilot.redirect
No tracked goals
OFF
ON
Save changes
Targeting
Rollout
Analytics
Settings
Include or exclude users based on key, country, IP, or other criteria
Include
User
paul-zaich ×
jeremy-reyes ×
james-dunbar ×
kaori-furusawa ×
patrick-cushing ×
member
true ×
admin_member
true ×
Add rule
Exclude
User
Add rule
InThePilot Redirect


The Haunted Codebase
LaunchDarkly

The Haunted Codebase: Flag Targeting UI

Oldest
Most complex
Most business-critical frontend code

The Problem

- 12 years of accumulated debt
- 66,000 lines of code across 400+ files
- Outdated patterns and libraries
- Unintuitive complexity of hundreds of individuals updating over an extended period of time.

The Cost

- Recent team wanted to make changes to the rollout menu (one of our most-used features)
- Abandoned effort entirely because iterating on our old code was too difficult.
- Engineers were spending weeks on changes that should have taken days or even hours.

Challenge: Rewrite Entire Flag Targeting Frontend

100% functional and visual parity – pixel-perfect feature equivalence

- 2 Engineers + Claude Code
- 6 weeks
- $10K inference budget
- No disruption for customers
- No scope creep!


Walk Before You Run
LaunchDarkly

Walk Before You Run

Make sure the codebase is “agent-ready” before taking on something ambitious.

Critical context

- Move dev docs into repo
- Define “good” and “bad” patterns
- Encode standards + patterns

Faster feedback

- 75% faster startup (Webpack → RSPack)
- 95% faster lint (ESLint → Oxlint)
- 50% faster type checking (Native TS)

Better guardrails

- Bugbot: 1,500+ PRs reviewed
- Meticulous: only 2 UI incidents since adoption
- Branch previews for frontend changes

> The things that make AI agents effective are the same things that make human developers effective.

Exorcising the Demons: The Plan

1

Week 1: Plan

Use agents to exhaustively document behavior + plan. Add a feature flag.

2

Week 2: Systems + Guardrails

Build out acceptance criteria + “agentic infrastructure”

3

Week 3: Build + Verify

I thought implementation would take 3 days

4

Week 4: Done?

Secretly I really thought we could do it in 4 weeks

5

Week 5: Cleanup

Delete the old code. Take a victory lap.

6

Week 6: Go on vacation?

Or maybe apply our learnings to another area?

LaunchDarkly

Exorcising the Demons: The Reality

Week 1: This is pretty big
Hundred of user stories. Dozens of flags.
Tens of thousands of lines of code

1

2

Week 2: We're almost there!
Every week I said "next week we're just going to build the whole thing"

Week 3: I thought we'd be farther
Bottlenecked on human bandwidth

3

4

Week 4: The autonomy trap
The allure of the perfect system is strong.

Week 5: "Incremental" progress
Still thousands of lines of code per "phase".
And the system mostly works.

5

6

Week 6: Still not done
36,000 LOC generated in 2 weeks but nowhere close to done. Turns out vibe-coding isn't fun forever

LaunchDarkly


The Autonomy Trap
LaunchDarkly

The Autonomy Trap

Rube Goldberg development

Chasing autonomous loops is very tempting, but is elusive and can come at the expense of your primary goal.

Human steering is a force multiplier

There are so many micro-decisions to make, the agent is almost guaranteed to make some that run counter to some of your unstated goals.

Maintaining focus is hard

Multi-tasking is great, until it’s not.
Human bandwidth is finite.

Friction is signal, not noise

The agent pushes through friction that would make a human pause and rethink. You never feel it.


Remodeling room by
LaunchDarkly

Remodeling room by room

More incremental, more human-in-the-loop, but still ambitious. Each “room” was still an entire feature – thousands of lines of code that would have taken us weeks to rewrite previously.

> **The key tactic:**
>
> Decompose the monolithic rewrite into ~22 discrete, independently verifiable phases. Each with clear inputs, acceptance criteria, and independent validation. Take learnings and improve the system to pay them forward to future phases.

The Output

39,000+	380+	~~22~~ 34	~$7K
lines of TypeScript and CSS	files	implementation phases	total inference spend

2 engineers. ~~6~~ 8 weeks. Shipped internally, with customer rollout next.

> The moment I knew it was going to work? When I started getting confused about which version I was looking at.

LaunchDarkly

What We Learned

Structure matters MORE with AI

Garbage in, garbage out, but at unthinkable speeds. The agent needs quality inputs, context, guardrails, and feedback loops.

The autonomy trap is real

The most seductive mistake is full automation. The perfect feedback loop is elusive and human steering is extremely high value.

Bottlenecks don’t vanish, they move

Writing code is no longer the bottleneck. Review, infrastructure, and approval cycles are. Invest in tooling first.

Rethink the approach, not just tools

The true unlock isn’t velocity, it’s ambition. But our old ways of thinking will hold us back.

Our legacy code isn't a liability.
It's our testing environment.

> AI didn't change our mission, it changed what's possible.

Every company in this room has code they're afraid to touch.
That's exactly where AI has the most impact: not greenfield projects, but the difficult, unglamorous, high-stakes work of modernizing a “mature” codebase.

LaunchDarkly


Q & A
LaunchDarkly